

16 — Fonctionnement détaillé : Le paiement Stripe

Explication du fonctionnement réel (tel que codé), pour répondre à l'oral « comment marche le paiement ? ».

En une phrase

On n'encaisse **jamais** l'argent nous-mêmes : c'est **Stripe** qui gère tout. Notre backend Go prépare une « session de paiement » chez Stripe, on redirige l'acheteur vers **la page de paiement hébergée par Stripe**, il paie, puis Stripe nous **prévient** que c'est payé (via un *webhook*). Chaque vendeur a son propre compte Stripe connecté, et on prélève **5 % de commission** au passage.

Carte de test : 4242 4242 4242 4242 , date future quelconque, CVC quelconque.

Le vocabulaire Stripe (à connaître)

Terme	Ce que c'est chez nous
Clé secrète (<code>sk_test_...</code>)	Mot de passe du backend pour parler à Stripe. En mode test .
Stripe Connect (compte Express)	Chaque vendeur/pro crée un compte Stripe relié au nôtre → il reçoit l'argent directement.
Checkout Session	La page de paiement toute prête, hébergée par Stripe. On redirige le client dessus.
Application Fee	Notre commission de 5 % prélevée automatiquement sur chaque vente.
Webhook	Stripe appelle notre serveur pour dire « c'est payé » → on met la base à jour.

Fichiers concernés : `API/admin/stripe.go` (créer les paiements), `API/admin/webhook.go` (recevoir les confirmations), `API/route/stripe.go` (les routes), `Frontend/script/annonces/oneAnnonce.js` (bouton

Acheter).

Étape 0 — Le vendeur connecte son compte Stripe

Avant de pouvoir vendre, un utilisateur doit avoir un compte Stripe. La fonction `ConnectToStripe` (`stripe.go`) : 1. Crée un **compte Stripe Express** pour lui (`account.New`). 2. Enregistre son `stripe_account_id` dans la table `utilisateur` . 3. Renvoie un **lien d'inscription** Stripe où il remplit ses infos bancaires.

C'est ce compte qui recevra l'argent de ses ventes.

Cas n°1 — Acheter une annonce

Côté navigateur (`oneAnnonce.js`)

Quand on clique « Acheter maintenant », `openCheckout('buy')` fait juste :

```
const response = await fetch(
  `${API_BASE_URL}/api/payment-annonce?annonce_id=${currentItem.id}&buyer_id=${buyerId}`,
  { method: "POST" }
);
const data = await response.json();
window.location.href = data.url; // on part sur la page de paiement Stripe
```

Si l'acheteur n'a pas de compte Stripe configuré, le backend répond **403** et on le renvoie sur son profil.

Côté backend (`PaymentAnnonce` dans `stripe.go`)

1. Récupère le **prix**, le **titre** et le `stripe_account_id` du **vendeur** en base.
2. Calcule le montant en centimes et la **commission de 5 %** :

```
unitAmount := int64(prix * 100) // Stripe travaille en centimes
commission := (unitAmount * 5) / 100
```

1. Crée une **Checkout Session** qui dit à Stripe : « fais payer ce montant, verse-le au vendeur (`Destination`), et garde 5 % pour nous (`ApplicationFeeAmount`) ».
2. Renvoie l'URL de la page Stripe → le navigateur y redirige.

Après paiement, Stripe renvoie le client sur notre `SuccessURL` (`oneAnnonce.html?...&payment=success`).

Cas n°2 — S'inscrire à un événement payant

Même principe (une Checkout Session), mais en plus le backend (`CreateEventCheckoutSession`) **écrit tout de suite en base** une commande « en attente » :

```
// 1) on crée la commande dans `order`  
INSERT INTO `order` (id_acheteur, id_annonce, montant_total, commission, date_commande) VALUES (  
// 2) on crée le paiement au statut "pending"  
INSERT INTO paiement (id_commande, stripe_id, statut) VALUES (... , 'pending')
```

On ajoute aussi `id_event` et `id_user` dans les **metadata** de la session, pour les retrouver plus tard dans le webhook.

Cas n°3 — Abonnement Pro (paiement récurrent)

`CreateProSubscriptionHandler` crée un **produit** + un **prix mensuel** chez Stripe, puis une Checkout Session en mode `subscription` (au lieu de `payment`). Trois formules :

Plan	Prix / mois
Premium	25 €
Plus	45 €
Pro	99 €

L'étape clé : le WEBHOOK (webhook.go)

C'est **Stripe qui appelle notre serveur** (route `POST /api/stripe/webhook`) pour confirmer les événements. On **vérifie la signature** pour être sûr que c'est bien Stripe (`WebhookSecret`), puis selon le type d'événement :

Événement Stripe	Ce qu'on fait en base
<code>checkout.session.completed</code>	Inscrit l'utilisateur à l'événement (<code>INSERT inscription</code>) + passe le paiement en <code>succeeded</code>
<code>account.updated</code>	Marque le compte vendeur comme vérifié (<code>stripe_verif_completed = 1</code>)
<code>customer.subscription.deleted</code>	Enlève le statut premium (<code>est_premium = 0</code>) + notifie l'utilisateur

Pourquoi un webhook et pas juste la page de succès ? Parce que le client pourrait fermer son navigateur avant le retour. Le webhook, lui, arrive **toujours** : c'est la source de vérité pour dire « le paiement est vraiment passé ».

Le schéma complet (achat d'une annonce)

```
Acheteur clique "Acheter"
→ oneAnnonce.js : POST /api/payment-annonce
  → backend : crée une Checkout Session (prix + 5% commission + vendeur)
  → renvoie l'URL Stripe
→ navigateur redirigé vers la page de paiement Stripe
→ l'acheteur paie (carte 4242...)
  → Stripe verse l'argent au vendeur, garde notre commission
  → Stripe appelle notre WEBHOOK → on met la base à jour
→ Stripe renvoie l'acheteur sur SuccessURL
```

Questions possibles à l'oral

« **Où est stocké l'argent ?** » Jamais chez nous. Stripe encaisse, verse au vendeur sur son compte Stripe connecté, et nous reverse notre commission de 5 %. On ne manipule aucune donnée bancaire → c'est **Stripe** qui gère la sécurité (norme PCI).

« **Comment vous prenez la commission ?** » Avec `ApplicationFeeAmount` (5 % du montant) + `TransferData.Destination` (le compte du vendeur) dans la session. Stripe fait la répartition tout seul.

« **C'est en vrai argent ?** » Non, on est en **mode test** (clé `sk_test_...`). On utilise la carte de test `4242 4242 4242 4242`. Aucun euro réel ne circule.

« **Pourquoi les montants sont $\times 100$?** » Stripe raisonne en **centimes**. 25 € = 2500. D'où

```
int64(prix * 100) .
```

« **Comment vous savez qu'un paiement a réussi ?** » Par le **webhook** `checkout.session.completed` , pas par la page de retour (qui peut être fermée). On y met le paiement à `succeeded` dans la table `paiement` .

Est-ce que le code est « débutant » ?

Le paiement est **la partie la plus avancée du projet**, c'est normal : payer en ligne demande de passer par un service pro. Mais **notre** code reste lisible : - on utilise la **librairie officielle Stripe** (`stripe-go`) qui fait le gros du travail ; - notre logique se résume à : lire un prix en base → calculer 5 % → créer une session → renvoyer une URL ; - le webhook est un simple `if event.Type == "..."` avec une requête SQL à chaque cas.

Il n'y a **pas d'algorithme compliqué**, juste des appels à Stripe et des `INSERT / UPDATE` . À l'oral, on l'explique comme un enchaînement d'étapes (le schéma ci-dessus), pas comme du code obscur.